RAG Study Companion

Complete Technical Study Guide

Author	**Maninderjit Singh Bhullar**
Program	M.S. Computer Science — Arizona State University
Stack	Python · LangChain · pgvector · FastAPI · Next.js · Dagster · Docker
Purpose	Interview preparation and deep technical understanding

Table of Contents

1. What is RAG and Why We Built It
2. System Architecture — The Big Picture
3. PDF Ingestion and Chunking
4. Embeddings — Turning Text into Numbers
5. pgvector — The Vector Database
6. Basic Retrieval
7. Advanced Retrieval: Sentence-Window Strategy
8. Advanced Retrieval: Auto-Merging Strategy
9. The Query Router
10. Conversational Memory — Multi-Turn Dialogue
11. The LangChain LCEL Chain
12. FastAPI Backend

13. Next.js Frontend with Streaming
14. Dagster Pipeline Orchestration
15. Custom RAG Evaluation (LLM-as-Judge)
16. Docker and Containerization
17. One-Minute Project Pitch
18. Interview Questions and Answers
19. Challenges Faced and How I Overcame Them

1. What is RAG and Why We Built It

RAG stands for Retrieval-Augmented Generation. It is a technique that combines the power of large language models (LLMs) with the ability to search and retrieve information from your own documents. Instead of relying solely on what an LLM learned during training — which has a knowledge cutoff and no awareness of your personal materials — RAG dynamically fetches relevant content from your own database and feeds it to the LLM as context before generating an answer.

Why not just use ChatGPT directly?

If you ask ChatGPT about your specific lecture notes, textbook chapters, or course materials, it has no idea what those documents say. It can only respond based on its general training. RAG solves this by: (1) storing your documents as searchable vectors, (2) finding the most relevant parts when you ask a question, and (3) giving those parts to the LLM along with your question so it can answer accurately and specifically from your own content.

The core problem RAG solves: Hallucination

Without RAG, LLMs 'hallucinate' — they confidently generate plausible-sounding but incorrect answers when they do not know something. By grounding every answer in retrieved context from real documents, RAG dramatically reduces hallucination. If the answer is not in the retrieved chunks, our system is explicitly instructed to say 'I don't have enough information in your documents to answer that' — which is the honest and safe behavior.

2. System Architecture — The Big Picture

The RAG Study Companion is a full-stack AI system with seven distinct layers working together. Here is the complete flow from a user's question to a grounded answer:

Layer	Component	Technology	Purpose
1	Document Ingestion	PyPDFLoader + LangChain	Load and parse PDF files
2	Chunking	RecursiveCharacterTextSplitter	Break text into manageable pieces
3	Embedding	HuggingFace sentence-transformers	Convert text to 384-dim vectors
4	Vector Store	pgvector (PostgreSQL)	Store and search vectors by similarity
5	Retrieval	Sentence-Window / Auto-Merging	Find relevant chunks for a query
6	Generation	OpenAI GPT-3.5-turbo	Generate grounded natural language answer
7	Interface	FastAPI + Next.js	Serve and display responses to user

Data Flow Diagram:

```
PDF File → PyPDFLoader (18 pages) → RecursiveCharacterTextSplitter (25 chunks) → HuggingFaceEmbeddings (384-dim vectors) → pgvector DB → [stored permanently] User Question → Query Router → [sentence_window OR auto_merging] → pgvector similarity search → Top-K relevant chunks → Conversational Memory → GPT-3.5-turbo → Grounded Answer → FastAPI → Next.js UI
```

3. PDF Ingestion and Chunking

What is chunking and why do we need it?

LLMs have a context window limit — they can only process a certain number of tokens at once. A typical textbook chapter might be 10,000 words, but we can only send a few hundred to the LLM as context. Chunking is the process of breaking a large document into smaller, self-contained pieces so that we can (a) embed each piece separately, (b) store them in a searchable database, and (c) retrieve only the relevant pieces for a given question rather than the entire document.

What tool do we use: RecursiveCharacterTextSplitter

We use LangChain's RecursiveCharacterTextSplitter. The word 'recursive' is key — it tries to split on the most natural boundaries first: paragraphs (\n\n), then lines (\n), then sentences (.), then words ( ). This means chunks are split at logical boundaries rather than cutting words or sentences in half.

Our exact configuration:

```
chunk_size=512, chunk_overlap=50, separators=['\n\n', '\n', '.', ' ']
```

Why chunk_size=512?

512 tokens is roughly 300-400 words — enough to capture a complete concept or explanation without being so large that the embedding loses specificity. Smaller chunks (128-256) are too granular and miss context. Larger chunks (1024+) make the embedding too general and hurt retrieval precision.

Why chunk_overlap=50?

When we split at position 512, the next chunk starts at position 462 (512-50). This 50-token overlap ensures that concepts spanning a chunk boundary are captured in at least one complete chunk. Without overlap, a sentence like 'The key advantage of ReLU is...' might be split with 'The key advantage of ReLU is' in one chunk and 'that it avoids the vanishing gradient problem' in the next — making both chunks incomplete.

What happens after chunking?

Our PDF of 18 pages became 25 chunks. Each chunk is a Python LangChain Document object containing two things: page_content (the actual text) and metadata (which page it came from, the source filename). This metadata is stored alongside the vector and used later for citation and filtering.

4. Embeddings — Turning Text into Numbers

A computer cannot compare two pieces of text for meaning directly. Embeddings solve this by converting text into a list of numbers (a vector) where semantically similar text produces numerically close vectors. This is the mathematical foundation that makes semantic search possible.

Which model do we use and why?

We use sentence-transformers/all-MiniLM-L6-v2 from HuggingFace. This model was specifically trained to produce embeddings where sentences with similar meaning cluster together in vector space. 'MiniLM' means it is a smaller, distilled version of a larger model — fast enough to run locally without a GPU, while still producing high-quality 384-dimensional embeddings.

What does 384-dimensional mean?

Each piece of text becomes a list of 384 decimal numbers, for example: [0.023, -0.187, 0.441, 0.092, ...]. These numbers encode the semantic meaning of the text. Two chunks about 'activation functions in neural networks' will have vectors that are close together (high cosine similarity), while a chunk about 'Module 6 deadlines' will have a vector far from both.

Why HuggingFace instead of OpenAI embeddings?

OpenAI's text-embedding-ada-002 model produces 1536-dimensional embeddings and costs money per API call. Our HuggingFace model runs locally, is completely free, produces 384-dimensional vectors that are smaller and faster to search, and performs comparably for document retrieval tasks. The trade-off is slightly lower quality on very nuanced semantic distinctions — acceptable for a study companion use case.

How embeddings enable search:

When a user asks 'What is an activation function?', that question is also embedded into a 384-dimensional vector. pgvector then finds the stored chunk vectors that are mathematically closest to the question vector using cosine similarity. This is why you can ask 'What does sigmoid do?' and get back a chunk that says 'Common Activation Functions: Sigmoid maps any input to 0-1' — the concepts are semantically similar even though the exact words differ.

5. pgvector — The Vector Database

pgvector is a PostgreSQL extension that adds a native vector data type and efficient similarity search operations to a standard relational database. This is our persistent storage layer for all embeddings.

Why pgvector instead of Pinecone, Weaviate, or Chroma?

Dedicated vector databases like Pinecone are cloud-only and cost money. Chroma is file-based and not production-grade. pgvector runs inside PostgreSQL — a battle-tested, ACID-compliant relational database that every engineering team already knows. This means we get vector search AND standard SQL querying AND metadata filtering in one system, dockerized and fully under our control. For a portfolio project, this also demonstrates systems engineering maturity that dedicated vector DB users do not show.

What is stored in the database?

Each row in the langchain_pg_embedding table contains:

Column	Type	Content
id	UUID	Unique identifier for each chunk
collection_id	UUID	Groups chunks by document collection ('study_docs')
embedding	vector(384)	The 384-dimensional float array representing chunk meaning
document	text	The raw text of the chunk (page_content)
cmetadata	jsonb	Page number, source file, and other metadata

How does similarity search work?

pgvector supports cosine similarity search using the <=> operator. When a query embedding arrives, PostgreSQL computes the cosine distance between the query vector and every stored vector, then returns the k rows with smallest distance (most similar). With an IVFFlat index, this scales to millions of vectors efficiently. Our collection of 25 chunks is tiny, so no index is needed — but the architecture would support 10M+ chunks with proper indexing.

Data persistence:

Data persists as long as the Docker container exists. Running 'docker start pgvector-db' brings it back with all 25 embedded chunks intact. Running 'docker rm pgvector-db' destroys all data. For production, we would mount a Docker volume to persist data independently of the container lifecycle.

6. Basic Retrieval

Basic retrieval (in retriever.py) takes a user query, embeds it using the same HuggingFace model used during ingestion, and asks pgvector to return the top-k most similar chunks. We use k=4, meaning we retrieve the 4 most relevant chunks for any question.

This is implemented using LangChain's PGVector.as_retriever(search_kwargs={'k': 4}). The retriever is then passed to the LangChain chain so that retrieval and generation happen in one pipeline call.

Limitation of basic retrieval:

Basic retrieval returns the raw chunks exactly as split during ingestion. Since chunks are 512 tokens, some chunks may be cut mid-concept. Also, if two chunks from the same page are highly relevant, basic retrieval returns both separately with no awareness that they're neighbors. This is exactly what advanced retrieval solves.

7. Advanced Retrieval: Sentence-Window Strategy

Sentence-window retrieval is our first advanced strategy, used for factual and definitional questions ('what is', 'define', 'explain', 'how does'). It addresses the core limitation of basic retrieval: a chunk that precisely matches a query may be cut off at exactly the wrong point, missing the actual answer by a few sentences.

How it works step by step:

Step 1: Perform standard similarity search to find k=4 matching chunks.

Step 2: For each matching chunk, identify its page number from metadata.

Step 3: Perform an additional similarity search filtered to that specific page to find neighboring chunks.

Step 4: Combine the original chunk with its page neighbors into one expanded Document.

Step 5: Return the expanded documents with richer context.

Why this improves results:

Imagine the chunk 'Activation (step) function: Now that we have a basic idea...' is the best match for 'what is an activation function'. The actual definition and examples are in the next chunk on the same page. Basic retrieval misses those examples. Sentence-window retrieval grabs the neighboring chunks from page 7 and combines them, so the LLM receives both the context sentence AND the Sigmoid/ReLU examples it needs to give a complete answer.

The analogy:

Think of it like finding a relevant paragraph in a book. Basic retrieval gives you just that paragraph. Sentence-window gives you that paragraph plus the paragraph before and after it — because the surrounding context often contains the actual answer to your question.

Trade-off:

Sentence-window retrieval makes more database calls (one per matched chunk) and returns larger context windows. For our 25-chunk dataset, this is negligible. For a million-chunk dataset, you would optimize this with pre-computed neighbor indices.

8. Advanced Retrieval: Auto-Merging Strategy

Auto-merging retrieval is our second advanced strategy, used for broad and summary questions ('summarize', 'list all', 'overview', 'everything about'). It solves a different problem: when a broad question causes multiple chunks from the same page to be retrieved, returning them separately creates fragmented, repetitive context.

How it works step by step:

Step 1: Perform similarity search with k=6 (larger than sentence-window to capture more of the document).

Step 2: Group all retrieved chunks by their page number.

Step 3: For pages where 2 or more chunks were retrieved (merge_threshold=2), concatenate all those chunks into one merged Document.

Step 4: For pages where only 1 chunk was retrieved, keep it as-is.

Step 5: Return the merged documents, which are fewer but more complete.

Concrete example from our project:

When asked 'summarize everything in module 6', basic retrieval returned 6 chunks. Three of those chunks came from page 1 (Today's Agenda content) and two came from page 3 (Module 6 explanation content). Auto-merging detected this overlap and merged the page-1 chunks into one comprehensive agenda document and the page-3 chunks into one explanation document, reducing 6 fragmented chunks to 2 complete ones. The LLM then received coherent, complete context rather than the same sentence repeated three times.

Why this matters for summarization:

When you ask a broad question, the answer is distributed across many parts of the document. Returning isolated chunks means the LLM sees disconnected fragments. Merging chunks from the same page reconstructs the original document structure, giving the LLM the full picture it needs to generate a comprehensive summary.

Deduplication:

Both strategies include a deduplication step that uses the first 100 characters of each chunk as a fingerprint. If two chunks are near-identical (which happens with chunk_overlap=50), only one is returned. This prevents the LLM from receiving the same sentence multiple times in its context window.

9. The Query Router

The query router is the decision-making layer that automatically selects the best retrieval strategy for each question. Instead of always using one strategy, the system intelligently matches the question type to the most appropriate retrieval approach.

How routing decisions are made:

Question Pattern	Keywords Detected	Strategy Selected	Reason
Factual / Definitional	what is, define, explain, how does, what are	sentence_window	Precise match + surrounding context needed
Summary / Broad	summarize, overview, list all, everything about, what topics	auto_merging	Wide coverage across multiple pages needed
Default / Ambiguous	none of the above	sentence_window	Precision preferred over breadth by default

Why not always use one strategy?

Sentence-window is great for precision but overkill for summaries. Auto-merging is great for coverage but wastes the context window on precise factual questions. The router gives each question the right tool, improving both answer quality and efficiency. This is also a system design signal for interviewers — it shows you thought about the trade-offs and designed around them.

Future improvement:

The current router uses keyword matching, which is simple and fast. A production system would use an LLM-based classifier to categorize questions more accurately, especially for ambiguous phrasings like 'tell me about module 6' which could be factual or summary depending on intent.

10. Conversational Memory — Multi-Turn Dialogue

LLMs are stateless — every API call is completely independent with no memory of previous exchanges. Without memory, a user asking 'what is an activation function?' followed by 'what are some examples of it?' would get no answer to the second question because 'it' has no referent in a stateless system.

Our implementation: ConversationBufferWindowMemory

We implemented a custom ConversationBufferWindowMemory class (k=3) that stores the last 3 question-answer pairs in a Python list. Before each LLM call, we load this history and inject it into the prompt as a 'Chat History' section. After each response, we save the new exchange to the history list.

Why k=3 (window size of 3)?

Storing only the last 3 exchanges is a deliberate design choice. Each exchange adds hundreds of tokens to the prompt. GPT-3.5-turbo has a 4,096 token context window — if we stored 10 exchanges, we might overflow the context window, truncating either the history or the retrieved chunks. k=3 balances conversational coherence with context window efficiency.

How it appears in the prompt:

```
Human: what is an activation function AI: An activation function introduces non-linearity in neural networks... Human: what are some examples AI: Common examples include Sigmoid, ReLU, and Tanh... [Current question: which one is most commonly used today?]
```

With this history, the LLM understands that 'which one' refers to activation functions, and answers correctly that ReLU is the most commonly used modern activation function.

Why we implemented it manually instead of using a library:

LangChain's ConversationBufferWindowMemory was deprecated and moved between packages multiple times during development. Rather than fighting dependency conflicts, we implemented the same concept manually in 15 lines of Python. This actually demonstrates deeper understanding than just calling a library — you know what the library was doing internally.

11. The LangChain LCEL Chain

LangChain Expression Language (LCEL) is the modern way to compose LangChain components using the pipe operator (|). Our main chain in chain.py looks like this:

```
chain = ( {"context": retriever | format_docs, "question": RunnablePassthrough()} | prompt |
llm | StrOutputParser() )
```

Reading this left to right:

retriever | format_docs: the question goes to pgvector, top-4 chunks come back, format_docs joins them into one string.

RunnablePassthrough(): the original question passes through unchanged.

| prompt: the context and question fill the PromptTemplate slots.

| llm: the filled prompt goes to GPT-3.5-turbo.

| StrOutputParser(): the LLM response object is parsed to a plain string.

Why LCEL instead of the older RetrievalQA chain?

RetrievalQA is deprecated. LCEL is the current standard — it is more transparent (you can see every step), more composable (easy to add memory, routing, or streaming), and more performant (async-native). Using LCEL signals that you know current LangChain idioms, not just tutorials from 2023.

12. FastAPI Backend

FastAPI is our Python web framework that exposes the RAG chain as HTTP endpoints the Next.js frontend can call. It runs via Uvicorn, an ASGI server that supports async Python natively.

Endpoints:

Method	Endpoint	Purpose
GET	/health	Health check — returns {status: ok}
POST	/ask	Takes {question: str}, runs RAG chain, returns {answer: str}
POST	/upload	Takes a PDF file, saves to /tmp, runs ingest_pdf(), returns success message

CORS middleware:

We add CORSMiddleware allowing requests from http://localhost:3000 (the Next.js dev server). Without this, the browser would block all requests from the frontend to the backend due to the same-origin policy. In production, this would be set to the deployed domain.

Why FastAPI over Flask or Django?

FastAPI is async-native (built on Starlette and Pydantic), which means it can handle concurrent requests without blocking. It auto-generates OpenAPI docs at /docs. It has type validation built in via Pydantic models. It is the current industry standard for Python ML APIs. Flask would work but is synchronous by default and has no built-in validation.

13. Next.js Frontend

The frontend is a Next.js 14 application using the App Router, TypeScript, and Tailwind CSS. It provides a chat interface where users can type questions, see streamed responses, and upload new PDFs.

Key frontend features:

- Chat UI with user messages (right, blue) and assistant messages (left, dark grey).
- PDF upload button that sends files to the /upload endpoint and shows confirmation.
- Enter key submission — no need to click Send.
- Auto-scroll to the latest message using useRef and scrollIntoView.
- Loading state with 'Thinking...' animated text while waiting for the backend.
- Error handling that shows a friendly message if the backend is unreachable.

Why Next.js over plain React?

Next.js provides file-based routing, server components, and built-in optimization out of the box. For a portfolio project, Next.js is also industry-standard for React applications and demonstrates full-stack awareness. The App Router pattern (app/page.tsx) is the current Next.js 13/14 paradigm, showing up-to-date knowledge.

Why Tailwind CSS?

Tailwind's utility classes (bg-gray-950, rounded-2xl, flex-1, etc.) allow rapid UI development without writing separate CSS files. Every major tech company uses Tailwind or a similar utility-first framework. It also produces consistent, responsive designs with minimal code.

14. Dagster Pipeline Orchestration

Dagster is a data pipeline orchestration tool. Instead of running 'python ingest.py' manually, Dagster wraps each step of the ingestion pipeline in an observable, schedulable, retryable asset.

Our three Dagster assets:

raw_documents — loads PDF pages. Tracks: page_count, source_file.

chunked_documents — splits into chunks. Tracks: chunk_count, chunk_size, chunk_overlap.

vector_embeddings — embeds and stores in pgvector. Tracks: chunks_stored, embedding_model, collection.

What Dagster gives us that a plain script does not:

Visual lineage graph showing raw_documents → chunked_documents → vector_embeddings.

Run history with timestamps, duration, and success/failure status.

Event logs showing exactly what happened at each step.

Metadata tracking — you can see how many chunks were produced in each run.

One-click re-runs from the UI if something fails.

Schedulers and sensors to trigger ingestion automatically when a new file appears.

Why this matters for the resume:

Most ML portfolio projects are just Jupyter notebooks or scripts. Adding Dagster demonstrates that you understand production data engineering — the same tools used by data teams at Airbnb, Stripe, and Replit. An interviewer asking 'how would you productionize this?' has already been answered.

15. Custom RAG Evaluation (LLM-as-Judge)

Most RAG portfolio projects have no evaluation. Ours measures system quality using three metrics, scored by a second LLM call acting as an impartial judge. This is the LLM-as-judge evaluation pattern used in production ML systems.

Three metrics we measure:

Metric	What it measures	Our score
Faithfulness	Is the answer grounded in the retrieved context, or did the LLM make something up?	0.800
Answer Relevancy	Does the answer actually address what was asked?	0.840
Context Precision	Are the retrieved chunks relevant to the question?	0.800

Overall Score: 0.813 / 1.0

Why no ground truth answers?

Traditional NLP evaluation requires human-written reference answers for comparison (ROUGE, BLEU scores). LLM-as-judge avoids this by having a second LLM evaluate quality through reasoning — the same way a human would. This is the current state-of-the-art for evaluating open-ended generative systems and is used by teams at Anthropic, OpenAI, and major AI labs.

Why we built it from scratch instead of using RAGAS:

RAGAS (the standard RAG evaluation library) had severe dependency conflicts with our Python 3.13 environment and newer LangChain versions. Rather than downgrade the entire stack, we implemented the same three metrics manually. This actually demonstrates deeper understanding — we know what RAGAS does under the hood.

16. Docker and Containerization

Docker is used to run our PostgreSQL + pgvector database in an isolated container. We use the ankane/pgvector image, which is PostgreSQL 15 with the pgvector extension pre-installed.

Our Docker command:

```
docker run -d --name pgvector-db -e POSTGRES_PASSWORD=password -e POSTGRES_DB=studydb -p 5432:5432 ankane/pgvector
```

What each flag means:

-d: run in detached mode (background). The container keeps running after the terminal closes.

--name pgvector-db: give the container a name so we can reference it with docker start pgvector-db.

-e POSTGRES_PASSWORD=password: set environment variable inside container for the DB password.

-e POSTGRES_DB=studydb: create a database called studydb on startup.

-p 5432:5432: map port 5432 inside the container to port 5432 on your Mac, so Python can connect.

Why Docker instead of installing PostgreSQL directly?

Direct installation pollutes your system with database processes, service files, and config files that are hard to clean up. Docker isolates everything — one command to start, one command to stop, and 'docker rm pgvector-db' completely removes it. Docker also makes the project reproducible — anyone can run the same command and get an identical environment regardless of their OS or existing software.

17. One-Minute Project Pitch

When asked 'tell me about your project' in an interview, use this structured 60-90 second pitch. Practice it until it feels natural — not memorized.

> "I built a full-stack AI study assistant called RAG Study Companion. The core idea is that students can upload their own PDF lecture notes or textbook chapters, and then ask natural language questions about that material — and get accurate, grounded answers instead of hallucinated ones. Under the hood, the system uses a RAG architecture. When a PDF is uploaded, I chunk it into 512-token pieces using LangChain's RecursiveCharacterTextSplitter, embed each chunk into 384-dimensional vectors using a HuggingFace sentence-transformers model, and store those vectors in pgvector running in Docker. What makes this project stand out is the retrieval layer. I implemented two advanced strategies — sentence-window retrieval for factual questions, which expands context around matched chunks, and auto-merging retrieval for broad questions, which consolidates chunks from the same page to avoid fragmentation. A keyword-based query router automatically selects the right strategy per question. I also added conversational memory so users can ask follow-up questions naturally, a Dagster pipeline for observable ingestion with a visual UI, and a custom LLM-as-judge evaluation suite that scored our system at 81.3% across faithfulness, answer relevancy, and context precision. The frontend is Next.js with Tailwind, talking to a FastAPI backend. The whole system is containerized with Docker. Happy to go deep on any part of the architecture."

18. Interview Questions and Answers

Q1: Why did you choose pgvector over a dedicated vector database like Pinecone or Weaviate?

pgvector runs inside PostgreSQL, which gives us vector search AND relational queries AND ACID compliance in one system that is dockerized and free. Pinecone is cloud-only and costs money. Weaviate adds operational complexity. For a production system where you need to join vector search results with user tables or filter by metadata using SQL, pgvector is the superior choice. It also signals systems engineering maturity — I understand that the database is infrastructure, not just a plug-and-play service.

Q2: What is the difference between sentence-window and auto-merging retrieval, and when do you use each?

Sentence-window retrieval is for precise, factual questions. When a chunk matches the query, we expand the context by pulling neighboring chunks from the same page — because the answer often sits just beyond the matching boundary. Auto-merging is for broad questions. When multiple chunks from the same page are retrieved, we merge them into one document to avoid fragmentation and repetition. The query router selects between them based on keyword patterns — 'what is' triggers sentence-window, 'summarize' triggers auto-merging.

Q3: How does your evaluation work, and why did you implement it yourself instead of using RAGAS?

We use LLM-as-judge evaluation — a second LLM call rates each answer on faithfulness (is it grounded in the retrieved chunks?), answer relevancy (does it address the question?), and context precision (were the right chunks retrieved?). We scored 81.3% overall. We implemented it manually because RAGAS had severe dependency conflicts with Python 3.13 and modern LangChain. This actually demonstrated deeper understanding — we know exactly what the metrics measure and how to compute them, rather than just calling evaluate() blindly.

Q4: What is chunk overlap and why is 50 tokens the right choice?

Chunk overlap means consecutive chunks share some tokens at their boundary. With chunk_size=512 and chunk_overlap=50, chunk 2 starts 50 tokens before chunk 1 ends. This prevents concepts that span a boundary from being split across two incomplete chunks. 50 tokens is roughly 30-40 words — enough to capture a sentence that bridges two chunks. Too low (10-20) and boundary concepts get split. Too high (200+) and you waste storage and retrieval quality by duplicating too much content.

Q5: How does the conversational memory work, and what is the trade-off in choosing k=3?

We implement a sliding window memory that stores the last k=3 question-answer pairs as a list of dicts. Before each LLM call, we format this history as 'Human: ... AI: ...' text and inject it into the prompt. After each response, we append the new exchange and drop the oldest if len > 3. k=3 is chosen because GPT-3.5-turbo has a 4,096 token context window. Each exchange is 200-400 tokens, the retrieved chunks are 500-1000 tokens, and the prompt template is 100 tokens. k=3 fits comfortably without truncation. k=10 would risk overflow and degraded answers.

Q6: Why use HuggingFace embeddings instead of OpenAI embeddings?

HuggingFace sentence-transformers/all-MiniLM-L6-v2 runs completely locally, is free, and produces 384-dimensional vectors. OpenAI text-embedding-ada-002 produces 1536-dimensional vectors and costs $0.0001 per 1K tokens. For a study companion ingesting hundreds of documents over time, local embeddings eliminate ongoing API costs entirely. The quality difference is minimal for document retrieval tasks. The trade-off is slightly slower embedding speed (seconds vs. milliseconds) — acceptable since ingestion happens once per document, not per query.

Q7: If you were to productionize this system, what would you change?

Several things: (1) Add user authentication so each user has their own document collection in pgvector. (2) Add a pgvector IVFFlat index for sub-millisecond similarity search at million-chunk scale. (3) Switch from keyword routing to LLM-based query classification for more accurate strategy selection. (4) Add a re-ranker (cross-encoder model) as a second-pass filter after initial retrieval to improve precision. (5) Mount Docker volumes for persistent storage independent of container lifecycle. (6) Add proper observability with Dagster sensors that auto-trigger ingestion when new files are uploaded.

19. Challenges Faced and How I Overcame Them

This section prepares you for 'what was the hardest part of this project?' — a common interview question. Answer these honestly and confidently.

Challenge 1: LangChain API Deprecations

Problem:	LangChain moved classes between packages multiple times between versions. Imports like langchain.document_loaders, langchain.text_splitter, and langchain.memory all broke as the library restructured into langchain-core, langchain-community, and standalone packages.
Solution:	I learned to read the deprecation warnings carefully — each one pointed to the new import path. I updated imports one by one: PyPDFLoader moved to langchain_community.document_loaders, RecursiveCharacterTextSplitter moved to langchain_text_splitters, and memory I re-implemented manually to avoid the dependency entirely. This taught me to be cautious about fast-moving libraries and to understand what each import actually does rather than copying tutorials blindly.

Challenge 2: RAGAS Dependency Conflicts with Python 3.13

Problem:	RAGAS internally imports ChatVertexAI from langchain_community regardless of whether you use it, and this import fails on Python 3.13 with newer LangChain versions. Every version of RAGAS conflicted with some part of our stack.
Solution:	Rather than downgrading Python or LangChain (which would break other things), I implemented the three RAGAS metrics manually using direct OpenAI API calls and LLM-as-judge prompting. This took 80 lines of code instead of 3 library calls, but I ended up with a cleaner, fully-understood evaluation suite. The 81.3% score we report is entirely our own code.

Challenge 3: Streaming Token Doubling in the Frontend

Problem:	When implementing streaming responses from FastAPI to Next.js using LangChain's astream(), every token appeared twice in the frontend — words like 'activation activation function function'. This was caused by LangChain's astream() emitting both individual tokens AND accumulated text chunks simultaneously.
Solution:	After investigating, I switched from streaming to a single synchronous API call that returns the complete answer as JSON. This eliminated the doubling entirely. The trade-off is that users see a brief loading state instead of character-by-character streaming, which is acceptable and actually more reliable. Proper streaming can be added later using Server-Sent Events.

Challenge 4: GitHub Push Protection Blocking API Keys

Problem:	GitHub's secret scanning blocked our push because an earlier commit contained the OpenAI API key in the .env file. Even after adding .gitignore, the key was still in git history.
Solution:	I used git filter-branch to rewrite the commit history and remove the .env file from all past commits. I then force-pushed the cleaned history. Going forward, .env is in .gitignore and the project uses a .env.example file showing the required variable names without values. This was also a good reminder to never commit secrets — even in early development.

Challenge 5: Duplicate Chunks in Retrieval Results

Problem:	With chunk_overlap=50, some adjacent chunks were nearly identical. When both were retrieved for the same query, the LLM received the same sentence multiple times, wasting context window tokens and confusing the response.
Solution:	I added a deduplication step using the first 100 characters of each chunk as a fingerprint. Before returning results, we filter to unique fingerprints only. This reduced context repetition significantly and improved answer coherence.

Quick Reference: Why Each Technology Was Chosen

Technology	What it does	Why we chose it
RecursiveCharacterTextSplitter	Chunks documents	Splits on natural boundaries (paragraphs > sentences > words)
all-MiniLM-L6-v2	Embedding model	Free, local, 384-dim, strong semantic similarity performance
pgvector	Vector store	PostgreSQL-native, free, dockerized, SQL + vector in one system
LangChain LCEL	Chain composition	Modern, async-native, composable, industry standard
Sentence-Window Retrieval	Precise retrieval	Expands context around matched chunks for factual Qs
Auto-Merging Retrieval	Broad retrieval	Consolidates page chunks to avoid fragmentation for summaries
Query Router	Strategy selection	Matches question type to optimal retrieval strategy
Conv. Buffer Memory	Multi-turn dialogue	Sliding window k=3 balances coherence with context limits
FastAPI + Uvicorn	Backend API	Async, type-safe, auto-docs, industry standard for ML APIs
Next.js + Tailwind	Frontend UI	File-based routing, modern React patterns, rapid styling
Dagster	Pipeline orchestration	Visual lineage, run history, schedulable, production-grade
LLM-as-Judge Eval	Quality measurement	No ground truth needed, state-of-the-art for generative AI eval
Docker	Containerization	Isolated, reproducible, no system pollution, one command setup
OpenAI GPT-3.5-turbo	Answer generation	Fast, cost-effective, strong instruction following

This document was generated as study material for interview preparation. Every concept described here was actually built and tested — not theoretical. The system achieved an 81.3% evaluation score across faithfulness, answer relevancy, and context precision. Good luck.